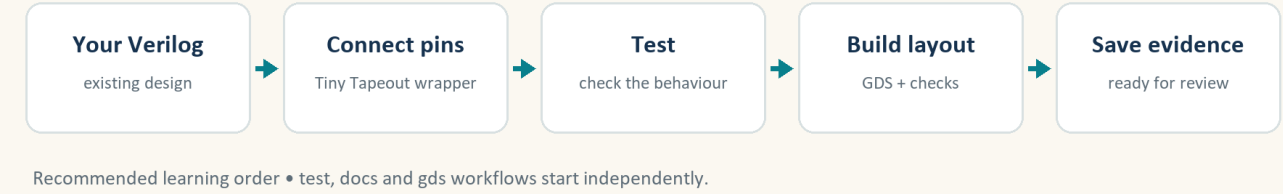
START HERE

Your Verilog → a checked chip layout

You already have a working Verilog design. This guide shows how to put it inside Tiny Tapeout, test the connections and build its layout. We use an eight-bit shift register throughout.



Recommended learning order • test, docs and gds workflows start independently.

01 Keep this guide beside your browser. Use the interactive HTML version for the clock experiment and tick each checkpoint as you finish.

02 Sign in to a free GitHub account. Download shift-register-starter.zip and extract it: on Windows, right-click → Extract All. Open the extracted folder containing info.yaml, src, test and docs. GitHub stores your project and runs the tools.

03 Follow pages 2–14 in order. Pages 13–14 cover gate-level simulation (GLS); page 15 is your final checklist and help sheet. You need no chip board or local tool installation for this browser route.

Tiny Tapeout = A service that combines small circuit designs on a shared chip. Its template includes the build tools.

Layout = The shapes and wires used to manufacture the circuit. GDS is the file format that stores them.

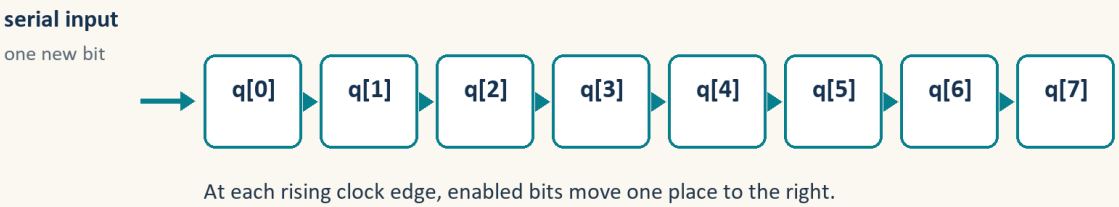
CHECK BEFORE YOU CONTINUE I have opened the guide, signed in to GitHub and found the starter ZIP.

Open GitHub / create a free account

KEEP YOUR CIRCUIT

1. Start with the design you already have

Our starting file is shift_register.v. It stores eight bits. Each rising clock edge moves the bits one position towards bit 7 and puts serial_in into bit 0, when enable is 1.



At each rising clock edge, enabled bits move one place to the right.

01 In the extracted folder, open src/shift_register.v with a text editor such as Notepad or VS Code. This is the reusable circuit; it does not contain Tiny Tapeout pin names.

02 Read its rule: reset wins; otherwise enable shifts; otherwise the stored value stays the same. Reset is synchronous: it takes effect at a rising clock edge.

03 Try the clock experiment in the HTML guide. From reset, shift in 1, then 0, then 1. Predict the eight-bit output after each edge.

src/shift_register.v

```
`default_nettype none

// Existing design: 8-bit serial-in, parallel-out shift register.
module shift_register (
    input  wire       clk,
    input  wire       rst,        // 1 clears q on the next rising clock edge
    input  wire       enable,     // 1 shifts; 0 holds the stored value
    input  wire       serial_in,  // new bit enters q[0]
    output reg  [7:0] q
);
    always @(posedge clk) begin
        if (rst)
            q <= 8'b00000000;
        else if (enable)
            q <= {q[6:0], serial_in};
    end
endmodule
```

RTL = The Verilog description of the circuit's behaviour and registers.

PAUSE & PREDICT Starting at 00000000, what do inputs 1, 0, 1 produce after three enabled edges?

Answer: 00000001 → 00000010 → 00000101. A disabled edge keeps 00000101.

CHECK BEFORE YOU CONTINUE I know my circuit's reset polarity, shift direction and enable behaviour.

BROWSER • GITHUB

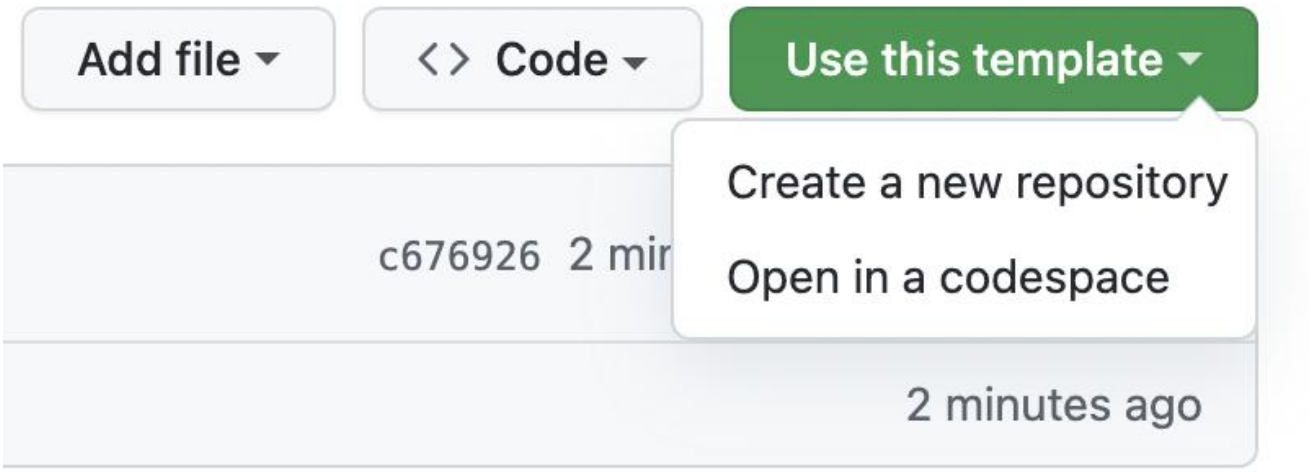
2. Make your own Tiny Tapeout project

A template is a ready-made set of folders and automation. A repository is your own saved project on GitHub.

01 Open the official HDL templates page below. Choose the SKY Verilog template used in this lesson. Confirm the repository is TinyTapeout/ttsky-verilog-template.

02 Sign in. Select Use this template → Create a new repository. Choose your own account as Owner.

03 Name it tt-shift-register, choose Public, and create it. Check that the new page begins with YOUR username. All later edits happen in this new repository.



GitHub's official screenshot shows the template menu. Use this control on the SKY template page.

CHECK BEFORE YOU CONTINUE My repository belongs to my account and contains src, test, docs and info.yaml.

Official Tiny Tapeout templates • SKY Verilog template

BROWSER • SETTINGS

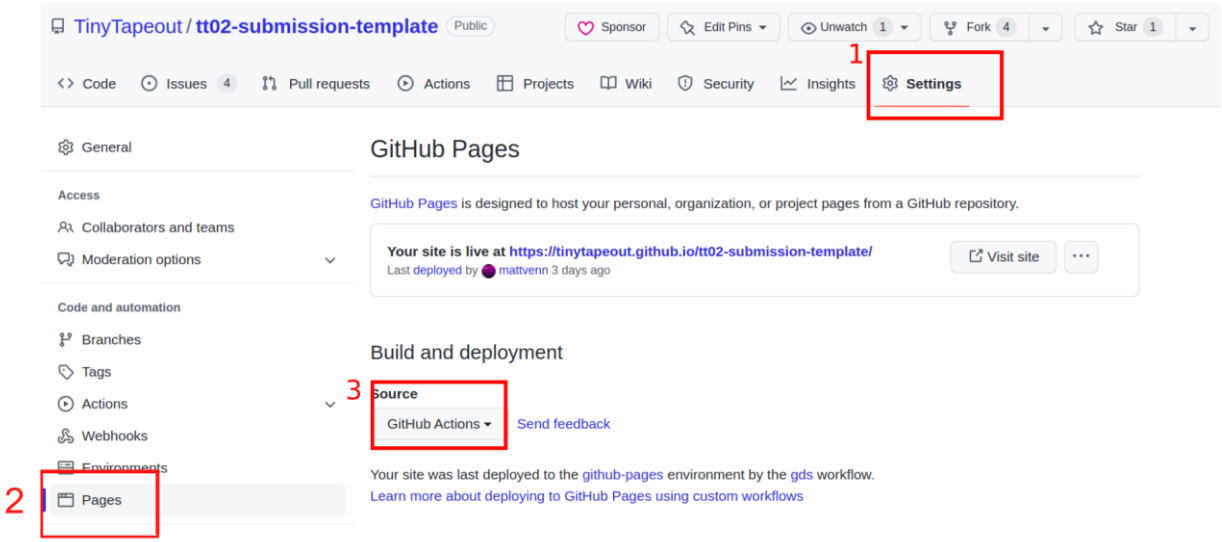
3. Turn on the build and results page

GitHub Actions runs the checks for you. GitHub Pages displays the finished layout in a browser.

01 In your new repository, open Actions. If you see an enable-workflows banner, enable it; otherwise continue.

02 Open Settings → Pages. Under Build and deployment, change Source to GitHub Actions.

03 Return to Code. Keep the template's .github folder, src/config.json and test/requirements.txt. The starter changes your design files, while these files supply the tools.



1 Settings → 2 Pages → 3 GitHub Actions. Official FAQ screenshot; step 2 annotation added.

Workflow = A saved recipe of jobs that GitHub runs, such as test or gds.

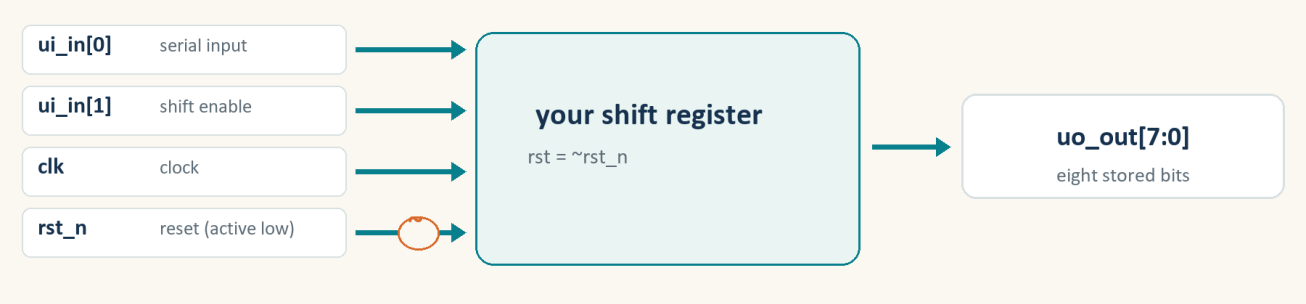
CHECK BEFORE YOU CONTINUE Actions is available and Pages → Source says GitHub Actions.

Pages troubleshooting

THE IMPORTANT ADAPTATION

4. Plan how your signals reach the chip pins

Your circuit has its own signal names. Tiny Tapeout expects a fixed set of ports. A wrapper is a small Verilog module that connects the two.



01 Use ui_in[0] for serial_in and ui_in[1] for the circuit's enable. Bits 2–7 are unused.

02 Connect clk directly. The platform's rst_n is active low: 0 requests reset. Our circuit's rst is active high, so connect rst to ~rst_n.

03 Send all eight q bits to uo_out. We do not use the bidirectional pins: set uio_out and uio_oe to zero. Keep all required wrapper ports.

Tiny Tapeout port	Our circuit signal	Meaning
ui_in[0]	serial_in	Next bit to shift in
ui_in[1]	enable	1 = shift; 0 = hold
clk	clk	Rising edge updates state
rst_n	~rst_n → rst	0 at platform = reset
uo_out[7:0]	q[7:0]	All eight stored bits

Port = A named input or output of a Verilog module.

Active low = The function is requested with 0 rather than 1.

PAUSE & PREDICT Is ena the same signal as our shift enable?

Answer: No. ena selects this project on the shared chip. Our shift enable is ui_in[1]. This core uses only ui_in[1] to decide whether to shift.

CHECK BEFORE YOU CONTINUE I can trace every used chip pin to a signal in my existing design.

EDIT • src/project.v

5. Add the wrapper around your design

Open the supplied project.v. It creates one instance of your shift register and connects the signals from page 5. The shift-register code itself stays reusable.

01 Use the complete supplied project.v as the replacement for the template's example.

02 For this first run, keep tt_um_student_shift_register unchanged everywhere. For a later submission, choose your own unique name beginning tt_um_ and change all three locations on page 7.

03 Notice that ~ means invert: ~rst_n converts the reset convention. uio_oe = 0 makes every bidirectional pin an input.

src/project.v

```
`default_nettype none

// Wrapper: connects the existing design to Tiny Tapeout's fixed ports.
module tt_um_student_shift_register (
    input  wire [7:0] ui_in,
    output wire [7:0] uo_out,
    input  wire [7:0] uio_in,
    output wire [7:0] uio_out,
    output wire [7:0] uio_oe,
    input  wire       ena,
    input  wire       clk,
    input  wire       rst_n
);
    shift_register core (
        .clk       (clk),
        .rst       (~rst_n),  // Tiny Tapeout reset is low; core reset is high
        .enable    (ui_in[1]),
        .serial_in (ui_in[0]),
        .q         (uo_out)
    );

    // Bidirectional pins are unused. A 0 in uio_oe means input direction.
    assign uio_out = 8'b00000000;
    assign uio_oe  = 8'b00000000;

    // ena is Tiny Tapeout's project-selection signal, not shift enable.
    wire _unused = &{ena, ui_in[7:2], uio_in, 1'b0};
endmodule
```

CHECK BEFORE YOU CONTINUE My wrapper has the eight required ports and instantiates shift_register with the intended connections.

Tiny Tapeout interface rules

EDIT • THREE NAMES, TWO LISTS

6. Tell both tools which files to use

The layout builder and simulator read separate source lists. Both must find your core and wrapper. The top module is the outermost circuit the tools should build.



01 In info.yaml, under project, use top_module: "tt_um_student_shift_register" and source_files: ["project.v", "shift_register.v"]. These filenames refer to files inside src.

02 In test/Makefile, set PROJECT_SOURCES = project.v shift_register.v. Use the supplied complete Makefile so the surrounding test setup remains intact.

03 In test/tb.v, the module instantiated as user_project must be tt_um_student_shift_register. The supplied tb.v already does this.

Location	Must agree
src/project.v	module tt_um_student_shift_register
info.yaml	top_module: tt_um_student_shift_register
test/tb.v	tt_um_student_shift_register user_project
info.yaml source_files	project.v + shift_register.v
test/Makefile PROJECT_SOURCES	project.v + shift_register.v

Testbench = Simulation-only code that connects the design to its test. It is not manufactured.

CHECK BEFORE YOU CONTINUE All three top-module names match; both source lists contain both Verilog files.

EDIT • info.yaml AND docs/info.md

7. Describe the design and its clock

The metadata identifies your project. The datasheet explains how someone can use its pins.

01 Open the supplied info.yaml. Replace the example author with your name. Keep meaningful title and description text, language Verilog, and the pin labels that match page 5. Keep every required pin key.

02 Open docs/info.md. Check its How it works, How to test and External hardware sections. It should say: reset clears on an edge; ui_in[1] enables shifting; ui_in[0] supplies the next bit.

03 For the first exercise keep the supplied 50 MHz target and the current SKY configuration's 20 ns clock period. The Python test uses 20 ns too. Metadata alone does not change the physical timing constraint.

Place	Clock setting
info.yaml	clock_hz: 50000000
src/config.json (template)	CLOCK_PERIOD: 20 (nanoseconds)
test/test.py cycle()	10 ns low + 5 ns to sample + 5 ns = 20 ns

Metadata = Settings and descriptions about your project.

50 MHz / 20 ns = 50 million clock cycles each second; one cycle lasts 20 nanoseconds.

CHECK BEFORE YOU CONTINUE My descriptions, pin map and clock settings agree with the actual design.

EDIT • test/test.py

8. Replace the example test with your test

The template's adder test cannot verify a shift register. Our Python test drives the wrapper's pins and checks the output after clock edges.

01 In the extracted folder, open test/test.py. Use this complete file to replace the template's test/test.py. cocotb is the Python tool that talks to the Verilog simulator.

02 Read the phases: reset, shift known bits, hold, shift more than eight bits so old data leaves, and reset while enable is 1. It also checks that unused outputs and bidirectional directions stay zero.

03 For your own design, change the expected result rule and input sequence to match its written behaviour. Keep checks at the external ports so the same test can run on the gate-level circuit.

test/test.py — expected-value rule

```
# Shift the previous value and add the new bit.
expected = ((expected << 1) | bit) & 0xFF
assert await cycle(bit) == expected
```

Simulation = Running a model of the circuit before making hardware.

Assertion = A check that stops the test when the observed result differs from the expected one.

PAUSE & PREDICT After shifting 1 eight times from reset, what should the output be? What if enable then becomes 0?

Answer: 11111111. With enable = 0 it stays 11111111, even when serial_in changes.

CHECK BEFORE YOU CONTINUE My test checks my circuit's behaviour rather than the template's example.

BROWSER • CODE

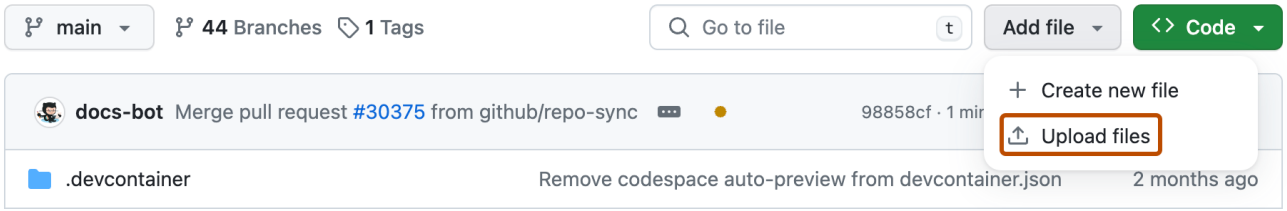
9. Upload the complete seven-file adaptation

Extract shift-register-starter.zip. Open its contents until you see info.yaml beside src, test and docs.

01 In your own repository's root, select Add file → Upload files. Drag info.yaml and the src, test and docs folders from INSIDE the extracted starter folder.

02 Check the paths: src/shift_register.v, src/project.v, test/Makefile, test/tb.v, test/test.py, docs/info.md and info.yaml. There must be no extra starter-folder prefix.

03 Enter Add shift register. For this personal practice repository choose Commit directly to main and save. A commit is a saved version. If GitHub requires a new branch, create a pull request and merge it into main. Check all seven files on main and note its commit identifier.



GitHub's official Add file screenshot locates the upload control. Upload into your repository root.

Folder dragging is unavailable → Open each existing file, select the pencil, replace its contents and commit. For src/shift_register.v use Add file → Create new file inside src. Check workflows only after all seven files are saved.

CHECK BEFORE YOU CONTINUE All seven files are saved at the intended paths, and the template infrastructure is still present.

BROWSER • ACTIONS → test

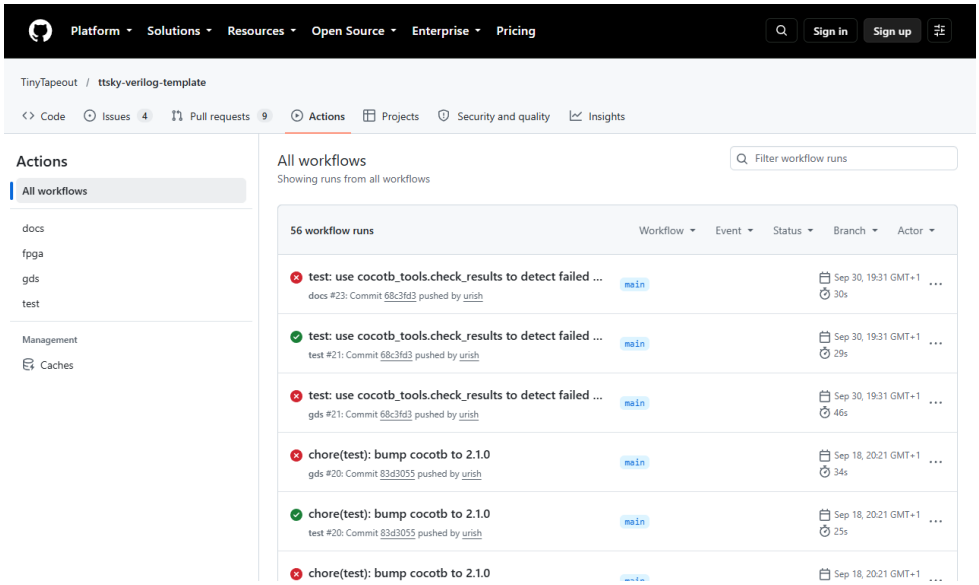
10. Check that your shift register passes

Each saved version can start checks. Open the run for the commit that contains your complete adaptation.

01 Select Actions → test → the run for your final commit. Confirm its branch and commit identifier.

02 Open the test job and expand Run tests. Read the final test summary: there must be no failures or errors. A green run from before your edit proves nothing about your new design.

03 If it fails, open the first assertion or compilation error. Fix that cause and save a new commit. The help sheet on page 15 covers common causes.



Official TEMPLATE Actions screen, for navigation only. Its red crosses are failed template runs, not your result. Your shift-register run has its own commit and result.

RTL test = A simulation of your Verilog source before it is converted into physical gates.

PAUSE & PREDICT What does a passing test establish?

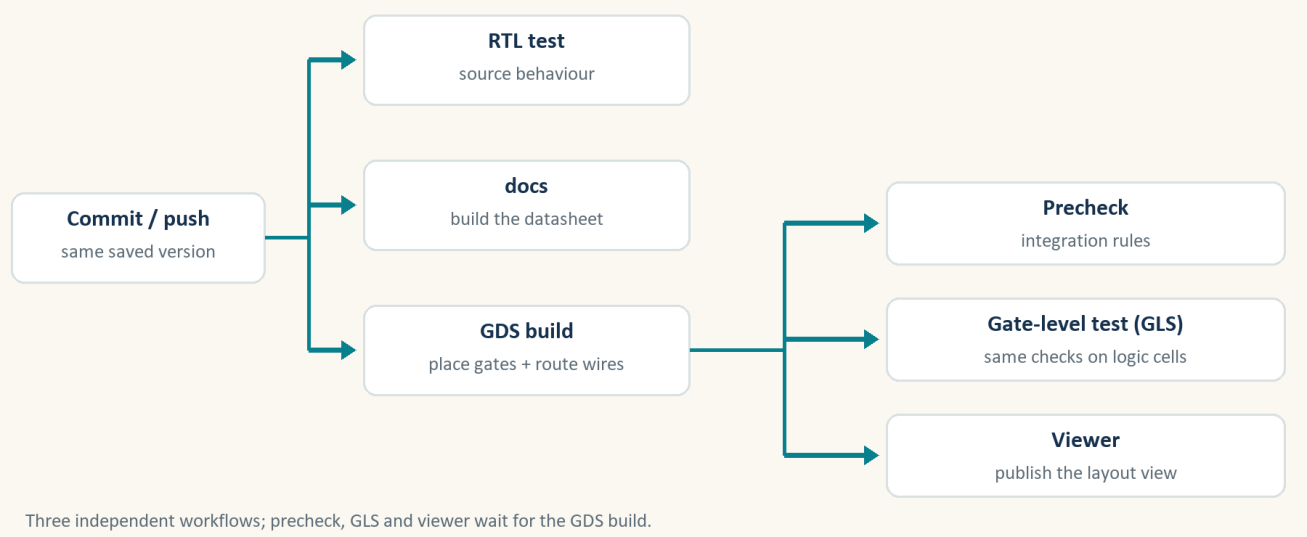
Answer: The adapted circuit passed the cases and assertions you wrote. It does not prove every possible behaviour, timing corner or manufactured chip.

CHECK BEFORE YOU CONTINUE The RTL test passed for my exact final commit, with zero failures and errors.

BROWSER • ACTIONS → gds / docs

11. Build, check and look at the layout

The gds workflow turns Verilog into physical gates and wires. The docs workflow builds your datasheet. These run separately from the RTL test.



Three independent workflows; precheck, GLS and viewer wait for the GDS build.

01 Open the docs workflow for the same commit. Confirm it succeeds. On its Summary page, scroll to Artifacts and download PDF; extract it and open datasheet.pdf. Check its pin table.

02 Open the gds workflow for that commit. Wait for the layout build, precheck, gate-level test and viewer jobs to complete successfully. Job labels can vary; open each job to see what ran.

03 Open the published Pages/viewer link from the run or repository Deployments. Zoom into the layout. On the gds run's Summary page, scroll to Artifacts: download tt_submission (layout package) and GDS_logs (reports). Artifact names may change in later templates.

Check	Plain meaning
Layout build	Place gates and connect them; inspect timing and geometry reports
Precheck	Check the design fits Tiny Tapeout's integration rules
Gate-level test	Run your test against the generated gate connections
Viewer	Publish an image/browser view of the layout

Artifact = A downloadable file produced by a workflow.

Gate-level = The circuit represented by the actual logic cells chosen during the build.

CHECK BEFORE YOU CONTINUE RTL, docs and every required gds job passed for one commit; I can open its layout and artifacts.

GLS • SAME TEST, GENERATED CIRCUIT

12. See what changes for gate-level simulation

Gate-Level Simulation (GLS) checks the circuit after the tools have converted your Verilog into logic cells. We run the same test at the same external pins.

01 Keep test/test.py checking ui_in, rst_n and uo_out, as supplied. Internal names such as core.q can disappear when the circuit is built; the chip's external ports remain the test interface.

02 Keep the GL_TEST sections in the supplied test/tb.v. Gate-level cells need power connections: VPWR is tied to 1 and VGND to 0 in the simulation fixture.

03 The gds workflow's gl_test job downloads the generated netlist and matching SKY130 cell models. It selects the Makefile's gate-level branch with GATES=yes. The cloud job supplies these dependencies; no local setup is needed.

RTL simulation	Gate-level simulation
Your source Verilog	Generated gate_level_netlist.v
Behaviour described in your code	Logic cells connected by the build
Same test/test.py	Same test/test.py
No extra cell power pins	GL_TEST fixture connects VPWR and VGND

test/tb.v — preserve this conditional connection

```
`ifdef GL_TEST
    .VPWR(VPWR),
    .VGND(VGND),
`endif
```

Netlist = A generated description of logic cells and the wires joining them.

CHECK BEFORE YOU CONTINUE My test uses the wrapper ports, and the supplied fixture and gate-level Makefile branch are intact.

Official SKY gate-level test action

BROWSER • ACTIONS → gds → gl_test

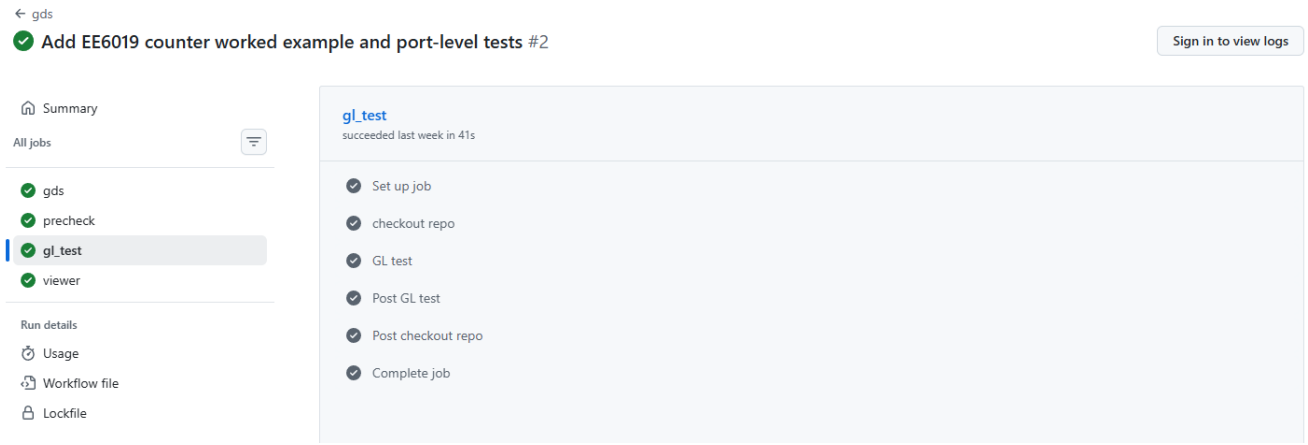
13. Open the GLS job and check its evidence

The template starts GLS automatically after the GDS build succeeds. A successful RTL test and a successful GDS build do not replace this separate test.

01 Open Actions → gds → the run for your complete commit. Select gl_test in Jobs. If it is waiting, let gds finish; if it was skipped, inspect the failed build first.

02 Expand GL test, then Run tests. Read the cocotb summary. Expect PASS with zero failed tests. For this supplied test, the summary should report TESTS=1, PASS=1, FAIL=0. The whole gl_test job must also finish successfully.

03 Return to the run's Summary page. Under Artifacts, download gatelevel_test_results. Extract it and open results.xml with a text editor: check for test results with no failure or error entries. Keep the supplied tb.fst or tb.vcd waveform too.



Genuine successful gl_test job from the earlier COUNTER example at commit 61ef06b. It shows where to open GL test; it is not a shift-register result.

PAUSE & PREDICT Does this GLS result prove that every timing requirement is met?

Answer: No. This is a functional gate-level test. Inspect the build's separate timing reports as well; this walkthrough does not run an SDF timing-annotated simulation.

RTL passes, but GLS fails → Read the first failed assertion. Check reset, the three top-module names and GL_TEST power connections. If necessary inspect the downloaded waveform at the failing time. Compare the same external pins and expected values used by the RTL test.

CHECK BEFORE YOU CONTINUE The gl_test job passed for the same commit; I saved its results XML and waveform.

Open the genuine counter GLS example

FINISH • CHECKLIST AND HELP

14. Keep your result and use the guide again

You have finished this exercise when your adapted circuit is tested and its checked layout is archived. Fabrication is a separate later step.

01 Save the repository URL, commit identifier, test/gds/docs run links, downloaded GDS/reports and datasheet together. Match every item to the same commit.

02 Use the checklist: pin map agrees; three names agree; two source lists agree; RTL and GLS pass; docs pass; all required gds jobs pass; layout and evidence are saved.

03 For another existing Verilog design, reuse this route: describe its behaviour, map its pins, wrap it, list its sources and write its tests. For an approved fabrication submission, choose the matching shuttle and follow the submission app's current checks.

Unknown module / file not found → Page 7: check both source lists, filename case and the three top-module names.

Reset or shift assertion fails → Pages 5 and 9: check reset polarity, ui_in bit numbers, shift direction and expected values.

docs fails → Page 8: complete title/author/description and the three datasheet sections; preserve YAML indentation and pin keys.

Viewer fails but layout succeeds → Page 4: set Pages Source to GitHub Actions. Follow the linked FAQ's rerun procedure.

Layout build or gate-level test fails → Open the first failing job's log. Read synthesis/timing/geometry warnings. Correct the cause; unchanged reruns will not repair the design.

CHECK BEFORE YOU CONTINUE I have saved a checked layout package for one identifiable commit.

Submission app — later, after approval • Official test guidance • Workflow and viewer help • Creating repositories from templates • Uploading files